

Lecture ÷ Arrays-1

Agenda

Kadane's Algorithm

Beggars outside temple

Rain water trapped

Qu-1 find max subarray sum.

-2	3	4	-1	5	-10	7
----	---	---	----	---	-----	---

ans = 11

-3	4	6	8	-10	2	7
----	---	---	---	-----	---	---

ans = 18

4	5	2	1	6
---	---	---	---	---

ans = 18

-4	-3	-6	-9	-2
----	----	----	----	----

ans = -2

Brute force approach

```
int maxSubArraySum(int[] A) {
```

```
    ans = -∞;
```

```
    for(i=0; i<n; i++) {
```

```
        for(j=i; j<n; j++) {
```

```
            subarray [i, j]
```

```
            sum = 0;
```

```
            for(k=i; k<=j; k++) {
```

```
                sum += A[k];
```

```
            }
```

```
            ans = max(ans, sum);
```

```
        }
```

```
    }
```

```
    return ans;
```

```
}
```

TC: $O(n^3)$
SC: $O(1)$

Approach 2

```
int maxSubArraySum(int[] A) {  
    pf[] = getPrefixArray(A);  
    ans = -∞;  
    for(i=0; i<n; i++) {  
        for(j=i; j<n; j++) {  
            subarray [i, j]  
            sum = 0;  
            if (i == 0) {  
                sum = pf[j];  
            } else {  
                sum = pf[j] - pf[i-1];  
            }  
            ans = max(ans, sum);  
        }  
    }  
    return ans;  
}
```

TC: $O(n^2)$
SC: $O(n)$

Try: carry forward

TC: $O(n^2)$
SC: $O(1)$

Approach 3

TC: $O(n)$ SC: $O(1)$

[Kadane's Algorithm]

Case 1: All no. in array are +ve.

4	5	2	1	6
---	---	---	---	---

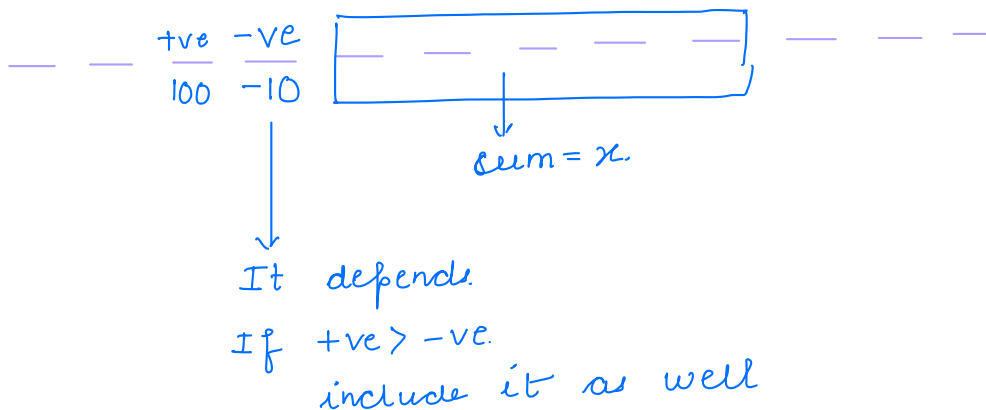
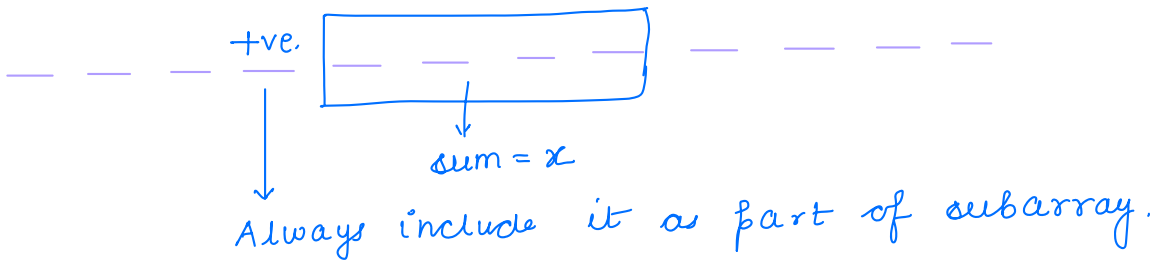
Ans = sum of array.

Case 2: All no in array are -ve.

-4	-3	-6	-9	-2
----	----	----	----	----

Ans = Max value of array

Case 3 Some no. are +ve and -ve



$A[] =$	5	6	7	-3	2	-10	-12	8	12	21	-4	7
sum 0	5	11	18	15	17	7	5 0	8	20	41	37	44
max $-\infty$	5	11	18	18	18	18	18	18	20	41	41	44

$A[] =$	-20	-10	-6	-15	-2	-30
$\text{sum} =$						
$\text{max} =$						

$A[] =$	-2	3	4	-1	5	-10	7	
sum 0	-2 0	3	7	6	11	1	8	
max $-\infty$	-2	3	7	7	11	11	11	

Algorithm

```
int maxSubArraySum(int[] A) {  
    sum = 0;  
    ans = -∞;  
    for (int el: A) {  
        sum += el;  
        ans = Math.max(ans, sum);  
        if (sum < 0) {  
            sum = 0;  
        }  
    }  
    return ans;  
}
```

TC: $O(n)$

SC: $O(1)$

Beggars outside temple { Amazon, Phonepe }

Given $arr[n]$. All elements are zero initially.

Given Q queries { index, value }.

Add this value starting from 'idx' till end of array.

Queries = 4

idx	value
1	3
4	2
2	1
1	-1

arr $\Rightarrow$

0	1	2	3	4	5	6
0	0	0	0	0	0	0
0	3	3	3	3	3	3
0	3	3	3	5	5	5
0	3	4	4	6	6	6
0	2	3	3	5	5	5

Brute force:

TC: $O(Q * n)$

SC: $O(1)$

Expected TC: Linear TC

Intuition:

	0	1	2	3	4	5	6
A =	0	0	x	0	0	0	0

pf[] $\Rightarrow$			+x	+x	+x	+x	+x
--------------------	--	--	----	----	----	----	----

idx	value
1	3
4	2
2	1
1	-1

	0	1	2	3	4	5	6
	0	0	0	0	0	0	0
	0	3	0	0	0	0	0
	0	3	0	0	2	0	0
	0	3	1	0	2	0	0
	0	2	1	0	2	0	0
pf:	0	2	3	3	5	5	5

Algorithm

```
void beggarsOutsideTemple(int[] A, int[][] queries) {  
    n = A.length;  
    q = queries.length;  
    for(i=0; i<q; i++) {  
        idx = queries[i][0];  
        val = queries[i][1];  
        A[idx] += val;  
    }  
    for(i=1; i<n; i++) {  
        A[i] = A[i] + A[i-1];  
    }  
}
```

TC: $O(q+n)$
SC: $O(1)$

Extension of Qu. 2

Given $arr[n]$. All elements are zero initially.

Given Q queries { start index, end index, value }

Add this value starting from 'idx' till end of array.

s_idx	e_idx	value
2	4	4
1	3	1
0	2	3
3	5	4

0	1	2	3	4	5
0	0	0	0	0	0
0	0	4	4	4	0
0	1	5	5	4	0
3	4	8	5	4	0
3	4	8	9	8	4

i	j	val
1	4	3
0	5	-1
2	2	4
4	6	3

0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0
0	3	3	3	3	0	0	0
-1	2	2	2	2	-1	0	0
-1	2	6	2	2	-1	0	0
-1	2	6	2	5	2	3	0

Approach

add 5

start index

end index

0	0	0	0	0	0	0
---	---	---	---	---	---	---

Previous approach $\Rightarrow$

5

5

5

5

5

5

-5

pf:

0

0

5

5

5

0

0

start index

end index

0	0	0	0	0	0	0
---	---	---	---	---	---	---

Add val
at start
idx

Add [-value]
at {end_idx + 1}

i	j	value
2	4	2
1	3	1
0	2	3
3	5	4

0	0	0	0	0	0

Algorithm

```
void beggarsOutsideTemple(int[] A, int[][] queries) {
```

```
    n = A.length;
```

```
    q = queries.length;
```

```
    for (i = 0; i < q; i++) {
```

```
        s_idx = queries[i][0];
```

```
        e_idx = queries[i][1];
```

```
        val = queries[i][2];
```

```
        A[s_idx] += val;
```

```
        if (e_idx + 1 < n) {
```

```
            A[e_idx + 1] -= val;
```

```
        }
```

```
    }
```

```
    for (i = 1; i < n; i++) {
```

```
        A[i] = A[i] + A[i-1];
```

```
    }
```

```
}
```

TC: $O(n+Q)$

SC: $O(1)$

Break: 8:09-8:19

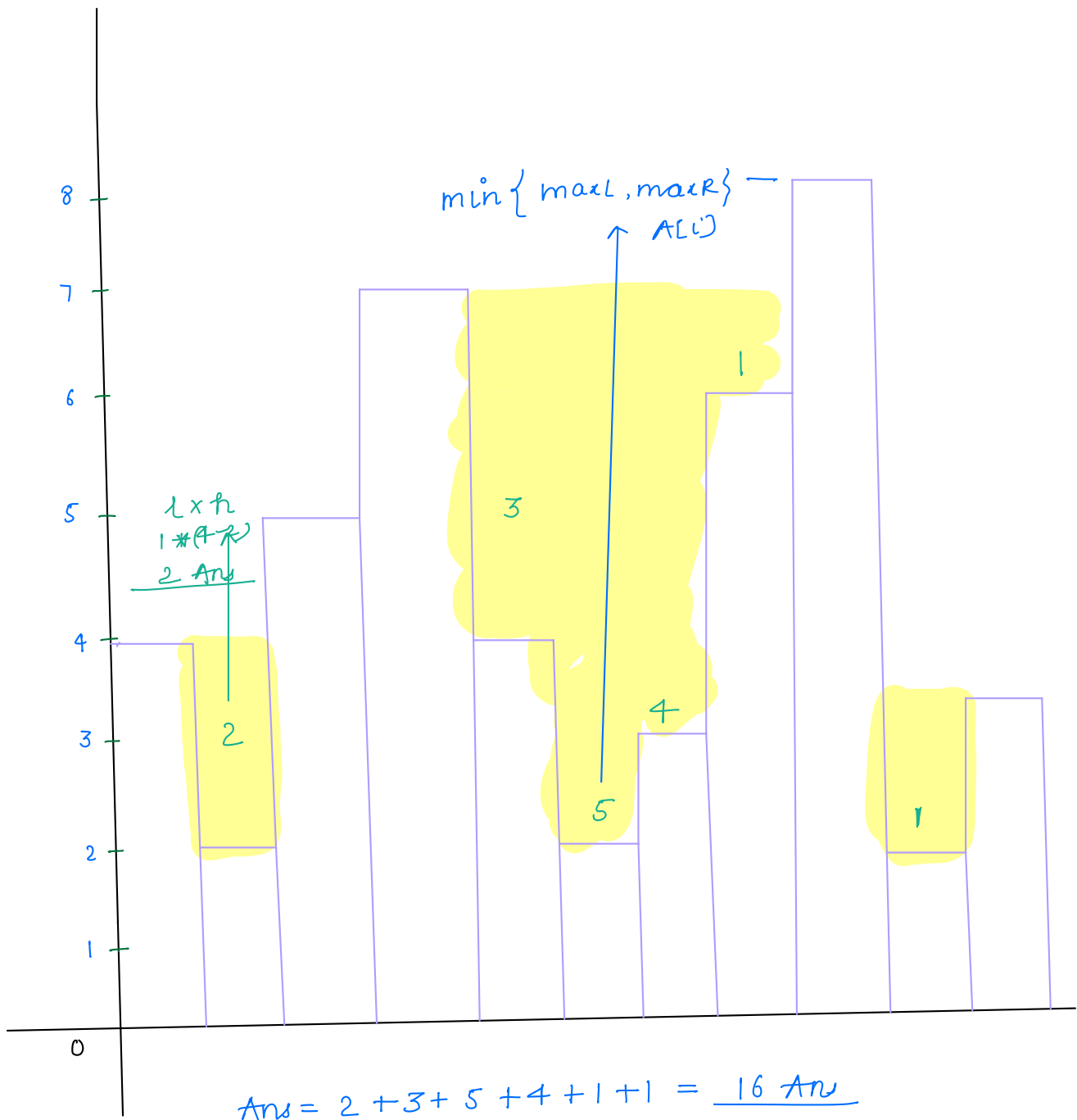
Rain water trapping [Interview]

Given n buildings with height of each building.

find rain water trapped b/w each buildings.

height[] =

4	2	5	7	4	2	3	6	8	2	3
---	---	---	---	---	---	---	---	---	---	---



Brute force approach

```
int totalWaterTrapped(int[] A) {
```

```
    ans = 0;
```

```
    O(n) — for(i=1; i<n-1; i++) {
```

```
        O(n) — maxL = getMaxLeft(A, 0, i-1);
```

```
        O(n) — maxR = getMaxRight(A, i+1, n-1);
```

```
        water = min(maxL, maxR) - A[i];
```

```
        if (water < 0) {
```

```
            water = 0;
```

```
        }
```

```
        ans += water;
```

```
    }
```

```
    return ans;
```

```
}
```

TC: $O(n^2)$

SC: $O(1)$

Approach 2

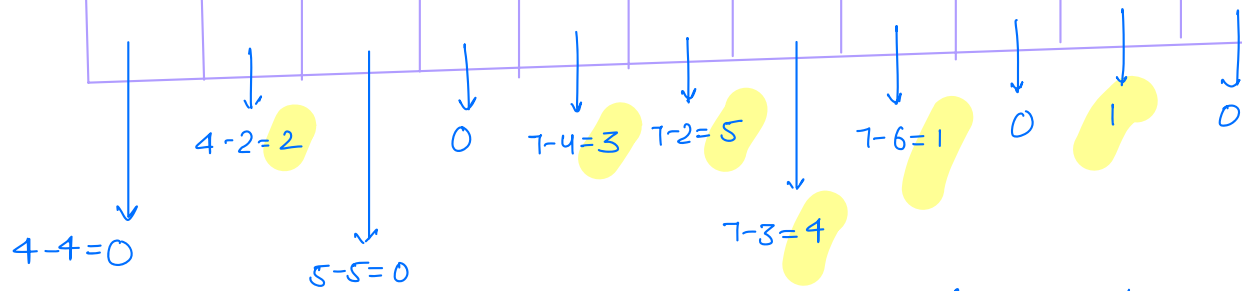
$$lmax[i] = \max[0, i]$$
$$rmax[i] = \max[i, n-1]$$

A[] =

4	2	5	7	4	2	3	6	8	2	3
4	4	5	7	7	7	7	7	8	8	8
8	8	8	8	8	8	8	8	8	3	3

lmax[] =

rmax[] =



Ans = 16 ↑

Algorithm

```
int totalWaterTrapped(int[] A) {  
    lmax[n]  
    rmax[n];  
    ans = 0;  
    lmax[0] = A[0];  
    for(i=1; i<n; i++) {  
        lmax[i] = max(A[i], lmax[i-1]);  
    }  
  
    rmax[n-1] = A[n-1];  
    for(i=n-2; i>=0; i--) {  
        rmax[i] = max(A[i], rmax[i+1]);  
    }  
  
    for(i=1; i<n-1; i++) {  
        water = min(lmax[i], rmax[i]) - A[i];  
        if(water < 0) {  
            water = 0;  
        }  
        ans += water;  
    }  
    return ans;  
}
```

TC: $O(n)$
SC: $O(n)$

Thankyou 😊